EMM

Esben's MIDI Mainframe

A Raspberry Pi that turns a USB hub full of synths into one patchbay, with network and Bluetooth MIDI built in.

User manual · version v0.1-alpha1-3-g4e333b1 · 2026-11-12

Source, issues and the technical documentation: the project's repository (see `README.md`).

This manual describes the software; where a statement has not been tried on real hardware yet, it says so.

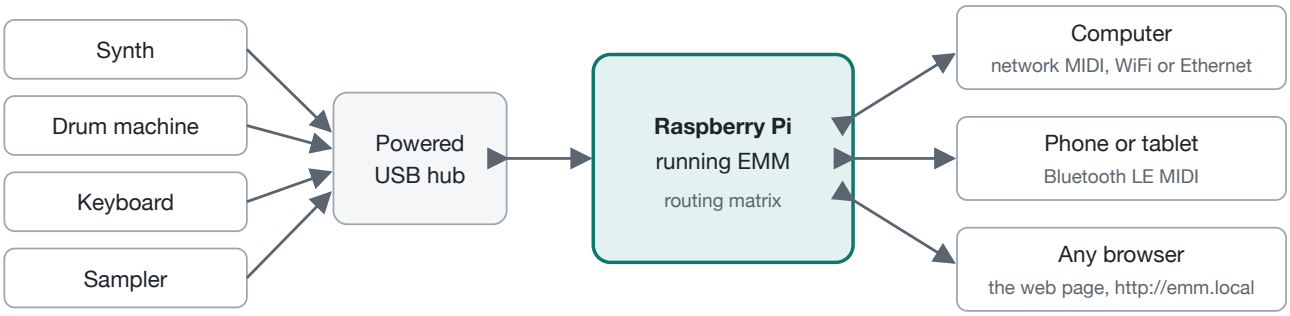
Contents

1. What EMM is
   - The idea in one picture
   - What you can do with it
2. What you need
   - The Pi, the hub, the power
3. Getting EMM onto a Pi
   - The appliance SD card
   - Raspberry Pi OS and the installer
4. First start and network setup
   - Ethernet, WiFi and the setup network
   - emm.txt
   - Starting over
5. The web interface
   - Matrix
   - Devices
   - Console
   - SysEx
   - Network
   - System
6. Connecting a computer, phone or tablet
   - Network MIDI (RTP-MIDI)
   - Bluetooth MIDI
   - What network MIDI is not for
7. Presets, backups and moving devices
   - Presets
   - Backup and restore
   - Moving a device to another hub port
8. Security
9. Troubleshooting
10. Reference
    - emm.txt keys
    - Files on the card
    - Limits and what is not included
    - Words used in this manual

1. What EMM is

The idea in one picture

You plug your synths, drum machines, keyboards and interfaces into a powered USB hub, and the hub into a Raspberry Pi. EMM sees every MIDI port and lets you connect any output to any input from a web page. Every USB device also appears on the network as a MIDI device of its own, so a laptop or tablet can talk to a synth without a cable.



How the parts fit together. Every USB device sends and receives MIDI through the hub, and also gets a network MIDI port of its own.

What you can do with it

- **Route MIDI between devices** with a matrix: any output to any input, several at once. Each connection can filter message types, limit channels, force a channel, or transpose notes. Connections survive unplugging and reboots.
- **Save the whole routing as a preset** and bring it back with one click.
- **Reach every synth from a computer over the network** (RTP-MIDI, also called AppleMIDI or network MIDI) for remote control, playing and loading a patch now and then.
- **Appear as a Bluetooth MIDI device** for phones and tablets.
- **Watch the MIDI traffic** live, and send MIDI by hand to a USB device: notes from an on-screen keyboard, the common messages, your own bytes.
- **Save and send SysEx dumps** from and to USB devices, as `.syx` files.
- **Back up and restore** the routing, presets and device names.

> **Know the limits of network MIDI**
> Network MIDI over WiFi or Ethernet is fine for playing music and for loading a patch now and then. Its speed and reliability are limited, so do not rely on it for backing up or restoring a device, or for firmware updates: use a USB cable for those. Section 6 explains why.

2. What you need

The Pi, the hub, the power

Board	Verdict	Notes
Raspberry Pi 4 B, Pi 400	Good	Gigabit Ethernet is separate from USB; dual-band WiFi.
Raspberry Pi 5	Good	More headroom. Needs the 27 W supply for full USB power.
Pi 500 / 500+, CM4, CM5	Probably good, untried	Same chips as the Pi 4 and 5. Compute modules need a carrier board with USB and may need USB host mode switched on, which the appliance card does not do: use Raspberry Pi OS for them.
Pi 3 B / B+	Works	Its Ethernet shares one USB 2.0 link with your MIDI devices (see the hardware notes in the repository). WiFi is not affected. 64-bit OS only.
Pi Zero 2 W	Works, with limits	One USB data port, so you need an OTG hub with its own power. 2.4 GHz WiFi only, which Bluetooth shares.
Pi 1, 2, Zero, Zero W	Not supported	32-bit only.

The USB hub matters most

- **Powered**, with a proper supply. The Pi cannot power several bus-powered devices.
- **USB 2.0 high-speed** or better. Prefer a hub with one transaction translator per port ("multi-TT"): with a single-TT hub all full-speed devices behind it share one 12 Mbit/s channel, which hurts when several synths send large SysEx dumps at once.
- Avoid very cheap hubs. Resets, glitches and power quirks look like "random" MIDI dropouts.
- Only **class-compliant** USB MIDI devices work. A device that needs a vendor driver on Windows or macOS usually is not one.

Power and cooling

- Pi 4: the official 5.1 V / 3 A supply. Pi 5: the official 27 W supply. A weaker supply limits the USB current, and an "under-voltage" warning is the first thing to rule out when devices disconnect.
- A Pi 4 or 5 in a closed box next to synths should be cooled; the System tab shows the CPU temperature.

> **WiFi, Bluetooth and timing**
>
> On 2.4 GHz the Pi's WiFi and Bluetooth share one antenna and disturb each other: use 5 GHz WiFi, or Ethernet. For tight timing, such as MIDI clock or fast live playing, use Ethernet: WiFi has no guarantees about delay and jitter. EMM turns WiFi power-save off, which removes tens of milliseconds of jitter.

3. Getting EMM onto a Pi

There are two ways. Both give the same EMM, with the same web interface and the same `emm.txt`. (The technical documentation in the repository calls the installer route "A" and the appliance card "B".)

	The appliance SD card	**Raspberry Pi OS and the installer**
What you do	Copy a set of files onto a FAT32 SD card	Flash Raspberry Pi OS Lite, run one command from your computer
What you get	A small "appliance" system that runs from memory, so the system itself is never written to. Only your settings are saved to the card: a power cut during a save at worst loses that save, and the card is checked and repaired at the next boot	A normal Debian system with EMM as a service; SSH, logs, `apt`
Needs	A copy of the project and `image/build.sh`, which builds `emm-sdcard-<version>.zip` (needs Docker, 15–30 GB of disk and one to two hours per board; see `docs/SDCARD-IMAGE.md`)	A copy of the project, a computer with Go 1.22 or newer and `ssh`, and Raspberry Pi Imager

The appliance SD card

1. Build or obtain `emm-sdcard-<version>.zip`. One zip works on the Pi 3 B/B+/A+, Zero 2 W, Pi 4/400 and Pi 5/500 families (compute modules: see section 2).
2. **Format the card as FAT32 with an MBR partition table.** The Pi's bootloader reads FAT32 only, and many cards over 32 GB come formatted as exFAT. The system is a few tens of megabytes, so a small card or partition is plenty.

   ```
   # macOS (erases the whole card: check the disk number with  diskutil list  first)
   diskutil eraseDisk FAT32 EMM MBRFormat /dev/disk4

   # Linux (replace sdX; erases the card)
   sudo parted /dev/sdX --script mklabel msdos mkpart primary fat32 4MiB 100%
   sudo mkfs.vfat -F 32 -n EMM /dev/sdX1
   ```

   On Windows the built-in formatter refuses FAT32 above 32 GB: use a card of 32 GB or less, or make a small partition with `diskpart` (check `list disk` first: `clean` erases the whole disk).

   ```
   diskpart
   list disk
   select disk N
   clean
   convert mbr
   create partition primary size=4096
   format fs=fat32 quick label=EMM
   assign
   ```

3. Unzip, and copy the **contents** of the `emm-sdcard-<version>` folder onto the card, so that `config.txt` is in the root of the card and not in a subfolder.

4. If you will use WiFi, copy `emm.txt.example` to `emm.txt` on the card and set at least `wifi_country` (section 4); the web interface has no field for it.
5. Put the card in the Pi, connect Ethernet (optional) and your powered USB hub, and power on.

> **What the card keeps**
> The card holds your hand-written `emm.txt` and an `emm` folder that EMM creates: `state.json` (routing, presets, device names) and `system.json` (network, web password and SSH, which are secrets). Pulling the plug is safe except in the second or so after you press *Save* in the web interface; that save may be lost. This behaviour has not yet been tested on a real card.

Updating: copy the new files over the old ones on the card and **keep** `emm.txt` and the `emm` folder.

Raspberry Pi OS and the installer

1. Flash **Raspberry Pi OS Lite (64-bit)**, Bookworm or Trixie, with Raspberry Pi Imager. In its settings turn on SSH, choose a user and password, and optionally set your WiFi and WiFi country. 32-bit images are not supported.
2. Boot the Pi on your network and check that `ssh pi@raspberrypi.local` works (with the user name you chose).
3. From a checkout of the project on your computer run

   ```
   scripts/deploy.sh pi@raspberrypi.local
   ```

   Add `--dry-run` first to see what it would do. It builds EMM for the Pi, copies it over and runs the installer there. The installer is safe to run again.
4. Open **http://emm.local/**. The installer renames a Pi that still has a default name to `emm`; if you chose another host name in Imager it keeps it, so use `http://<that name>.local/` (the installer prints the address at the end).

The installer sets up the system packages EMM needs, the EMM service, and a few small system settings (WiFi power-save off, no USB autosuspend for MIDI devices). It never overwrites your `emm.txt` or saved settings. **Updating:** run `scripts/deploy.sh pi@<the Pi's name>.local` again; settings stay. **Removing:** `scripts/deploy.sh --uninstall pi@…` keeps your settings and WiFi profiles so the Pi stays reachable.

4. First start and network setup

Ethernet, WiFi and the setup network

- **With Ethernet** (and DHCP on your network): open **http://emm.local/**.
- **With WiFi and nothing configured**: EMM opens its own network, **EMM-Setup-XXXX**. It is open (no password) unless you set `ap_password` in `emm.txt`. Join it with a phone or laptop. A "sign in to network" page opens by itself; if it does not, go to **http://emm.local/** or **http://10.42.0.1/**. Choose your WiFi, enter its password, and EMM joins it. Your phone drops off the setup network at that moment: reconnect to your normal WiFi and open **http://emm.local/**.
- **A Pi that is already online** (set up with Raspberry Pi Imager or `nmtui`) keeps its connection. EMM adopts it and does not open the setup network. Open `http://<the Pi's hostname>.local/` (the name it already has, for example `http://raspberrypi.local/`; EMM only renames the machine if you set `hostname`).
- If the saved WiFi cannot be joined within about 45 seconds at boot and Ethernet has no connection (wrong password, router gone), EMM opens the setup network again. While it is open it checks every 30 seconds whether Ethernet has come up.

Set the WiFi country (`wifi_country` in `emm.txt`, for example `DK`). In many regions WiFi stays off until it is set, and the 5 GHz channels you may use depend on it.

emm.txt

To start one, copy `emm.txt.example` to `emm.txt` and remove the `#` in front of the lines you want. It is a plain text file in the root of the card (`/boot/firmware/emm.txt` on Raspberry Pi OS) for settings you want fixed from the start: WiFi, hostname, passwords and so on. One `key = value` per line; a line starting with `#` is a comment. **Keys in `emm.txt` override the web interface and are applied on every boot**; leave a key out to manage it from the web page. The full list is in the reference (section 10). The file `emm.txt.example` on the card lists every key, commented out.

```
hostname      = studio
wifi_country  = DK
wifi_ssid     = MyNetwork
wifi_password = "correct horse #1"
web_password  = something-private
```

Starting over

To get the setup network back, create an empty file called `reset-network` next to `emm.txt` and reboot; EMM deletes it again, and leaves setup mode within about 30 seconds of an Ethernet cable being plugged in. To clear the routing, presets and device names, delete `emm/state.json` and any `emm/state.json.new` or `.old` next to it; to clear the network, web password and SSH settings, do the same for `emm/system.json`; then reboot. (On Raspberry Pi OS these files are in `/var/lib/emm/`; `sudo rm /var/lib/emm/state.json*` removes them.) A power cut can leave a `.new` or `.old` copy, and EMM would bring back the newest complete one.

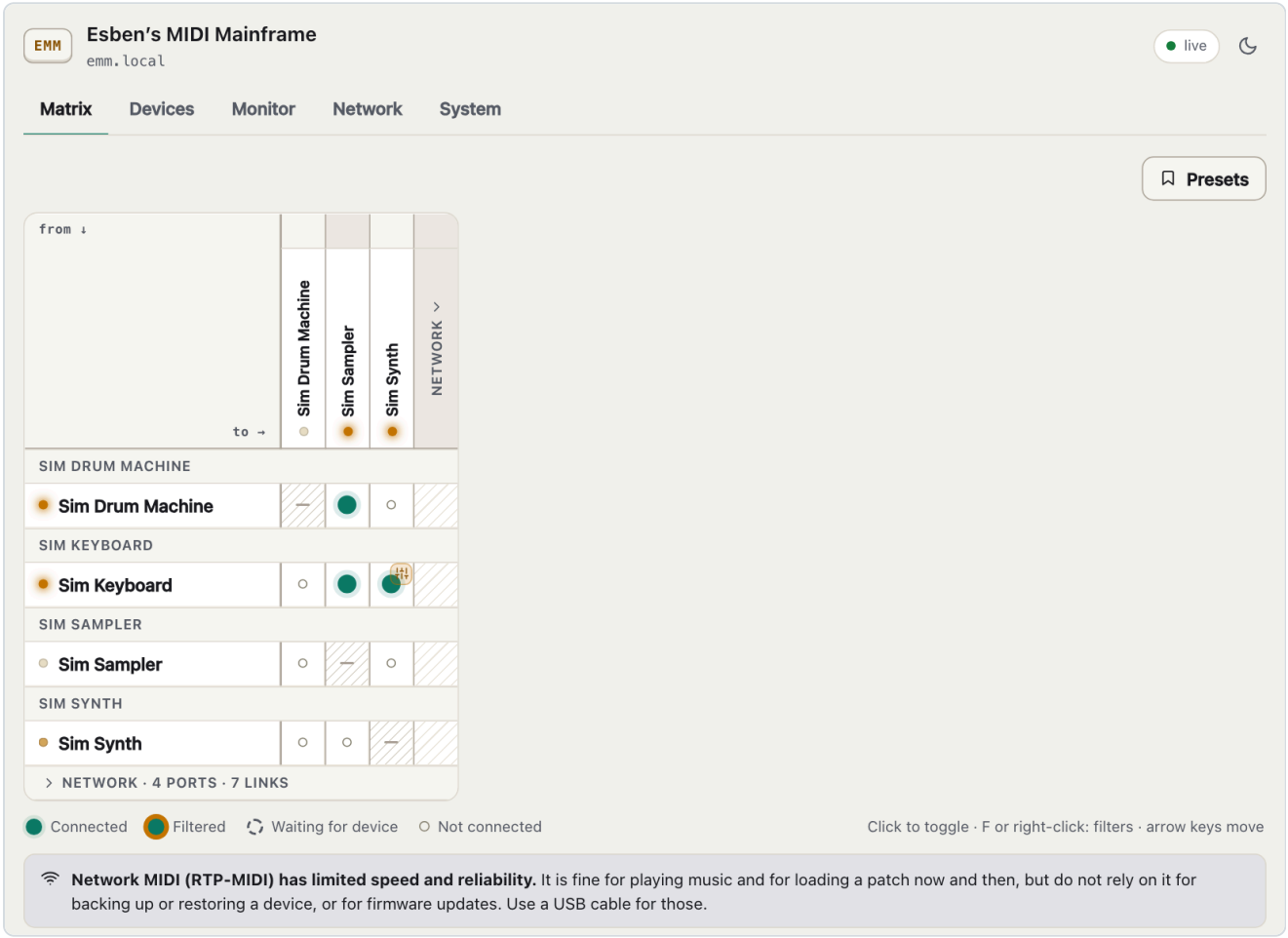
Nothing here removes what you wrote in `emm.txt`. Its WiFi, hostname and Ethernet lines win over the web interface at every boot. To change those, edit or remove the lines in `emm.txt` (card in a computer).

5. The web interface

Open **http://emm.local/** in any browser on the same network. The page has five tabs. The green "live" mark at the top right means the page is receiving updates from the device. The screenshots in this manual were taken with simulated devices.

Matrix

The matrix is the heart of EMM. **Rows are sources** (where MIDI comes from), **columns are destinations** (where it goes). Click a cell to connect or disconnect; a filled circle means connected. Where a row and a column are the same port the cell is hatched with a dash: a port cannot feed itself. Several sources can feed one destination, and one source can feed several.



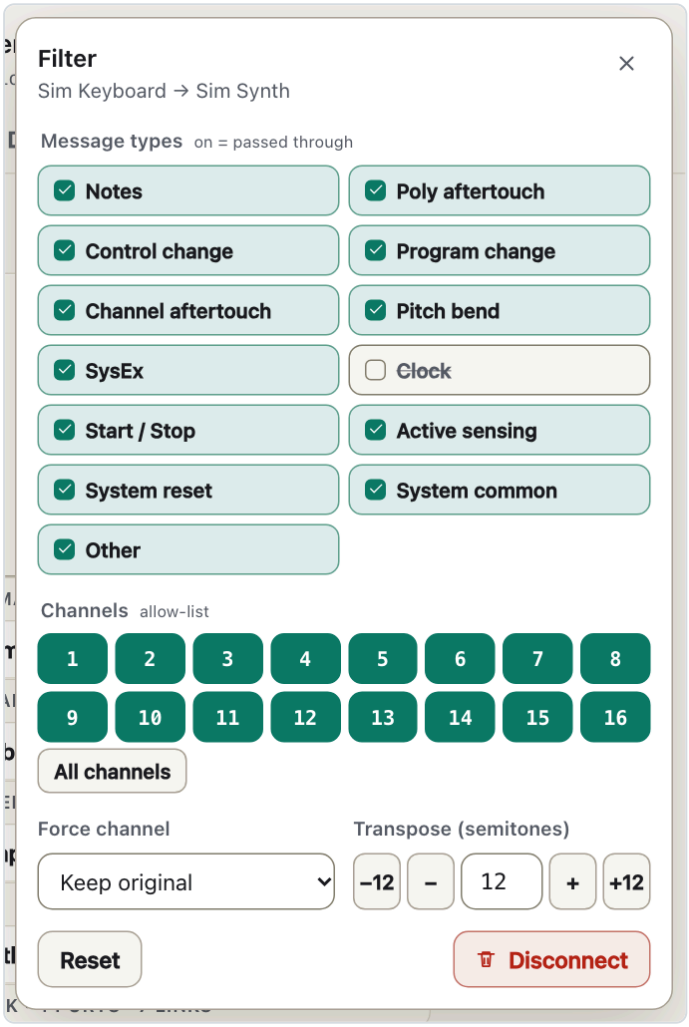
The Matrix tab. The keyboard feeds the sampler and (filtered) the synth; the drum machine feeds the sampler. The network ports are folded into the group at the bottom of the grid (its seven links are the automatic ones described below). Under the matrix: the legend and the note about network MIDI.

- **Legend.** Connected; *Filtered* (the connection changes or drops some messages); *Waiting for device* (the connection is remembered, but the device is not plugged in); *Not connected*.
- **The small dots** flash when MIDI arrives or leaves. The dot left of a source name flashes when that port receives MIDI from its device (even if nothing is routed from it); the dot under a destination's name flashes when MIDI is sent out to it, after the connection's filter.

- **Network ports** (the RTP-MIDI ports and the Bluetooth MIDI port) are folded into one group at the bottom. Click it to open the group. Each USB port with its *Network port* switch on is linked automatically, in both directions, to its own network port; these links count in the group's "links". They are ordinary connections: you can filter or delete them, and a preset recall that lacks them removes them. Switching the Network port off and on again restores them.
- **On a phone** the matrix turns into a list: pick a destination, then switch on the sources that should feed it.
- The keyboard works too: arrow keys move, `Space` or `Enter` toggles, `F` opens the filter.

Filters

Right-click a connected cell (or select it and press `F`) to adjust that one connection. On a touch screen, press and hold the cell, or tap the small sliders button that appears on a connected cell (or beside a switched-on source in the phone list). Changes take effect immediately; there is no Save button. Close the editor with the ×, `Esc` or by clicking outside it.



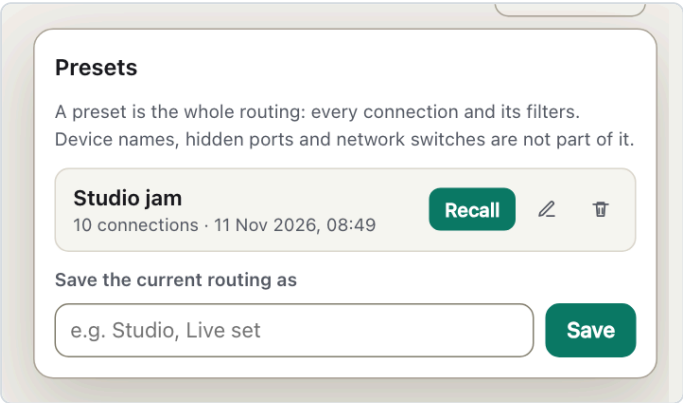
The filter editor. Here MIDI clock is blocked and notes are moved up one octave.

- **Message types.** Switch off the kinds of message that should *not* pass: notes, polyphonic or channel aftertouch, control change, program change, pitch bend, SysEx, clock, start/stop, active sensing, system reset, system common, other. A new connection passes everything except active sensing.
- **Channels.** All 16 are on by default; switch off the channels that should not pass. At least one must stay on (to silence a connection, switch off message types instead). *All channels* turns them all back on.

- **Force channel.** Puts every channel message onto one channel.
- **Transpose.** Shifts notes by up to 48 semitones either way. A note that would land outside the MIDI range (0 to 127) is dropped.
- **Reset** returns to the defaults; **Disconnect** removes the connection.

Presets

The *Presets* button above the matrix saves the **whole routing**, every connection with its filters, under a name, and brings it back later with *Recall*.



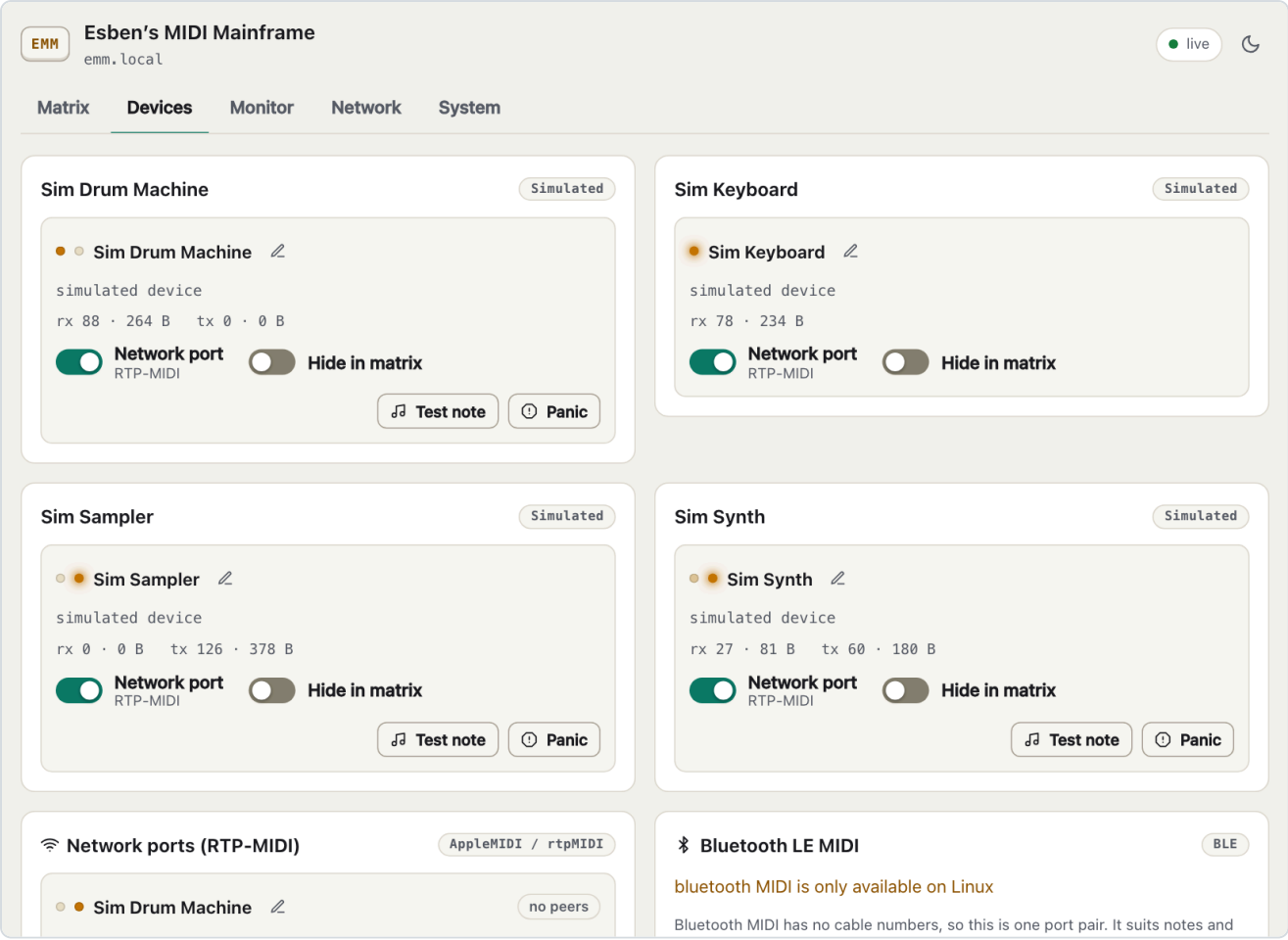
The presets menu.

Recalling a preset **replaces** the current routing. Device names, hidden ports and network switches are not part of a preset. If a preset names a device this unit has forgotten, that connection is skipped and the page tells you. Each preset also has *Rename* and *Delete* buttons. You can keep up to 50 presets, with names of up to 40 characters that differ from each other (capitals do not count); saving under an existing name asks before replacing it.

> **Network MIDI has limits**
>
> Below the matrix the page says: *Network MIDI (RTP-MIDI) has limited speed and reliability. It is fine for playing music and for loading a patch now and then, but do not rely on it for backing up or restoring a device, or for firmware updates. Use a USB cable for those.* See section 6.

Devices

One card per USB device, listing its ports. A device that has several MIDI cables shows several ports.

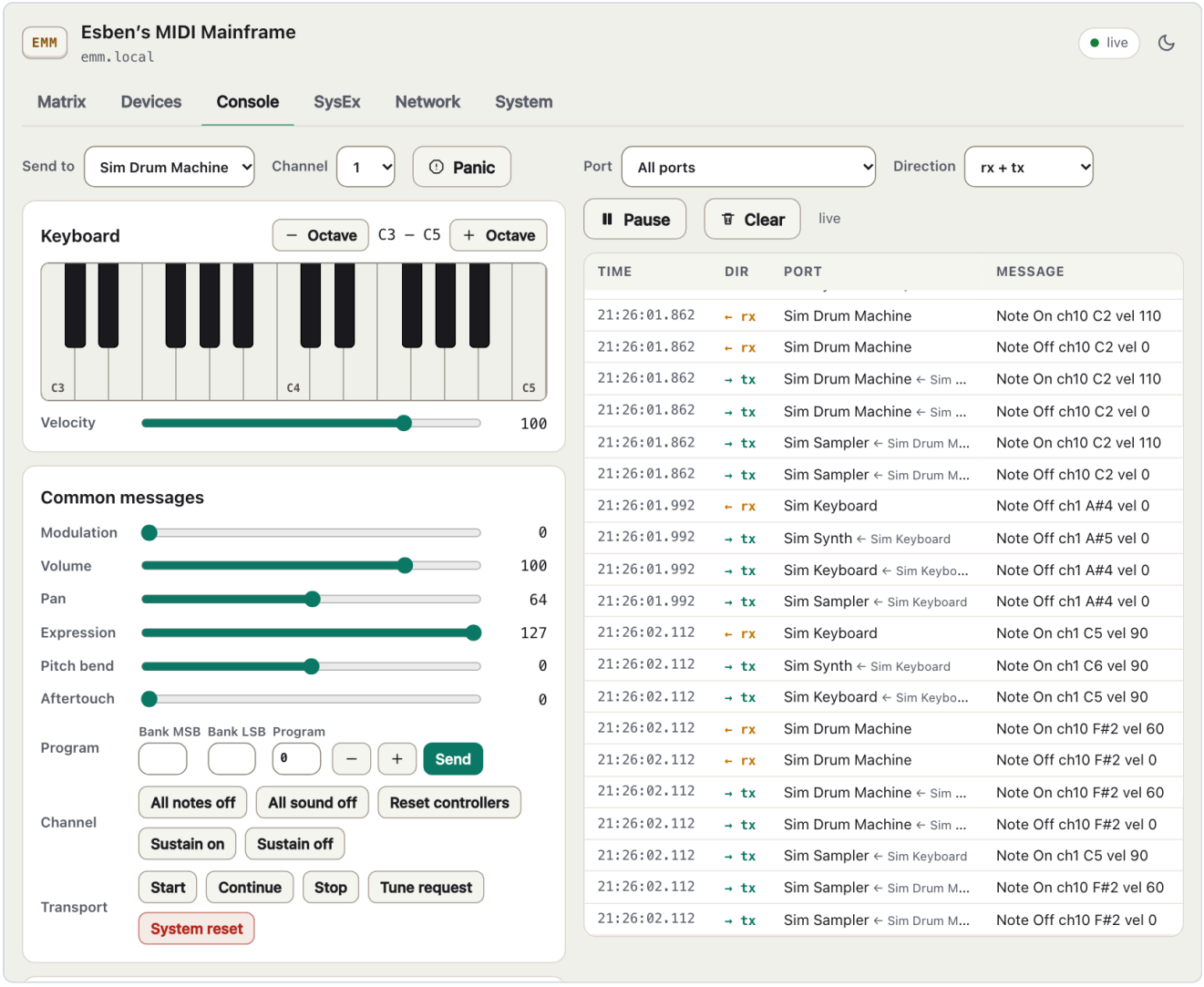


The Devices tab. The Bluetooth message on this development machine does not appear on a Pi, where the card shows whether Bluetooth is advertising and how many centrals are connected.

- **Rename** a port with the pencil (up to 60 characters). The name is used throughout the interface and, for a USB device's network port, it is also the session name a computer sees, so you may have to reconnect from the computer.
- **Network port** decides whether the device is also shown on the network as a MIDI device of its own. Switching it off removes its network connections.
- **Hide in matrix** takes a port out of the matrix without forgetting it.
- **Test note** (on devices that can receive MIDI) plays middle C on channel 1 for a third of a second, so you can hear that the cable works and, if the device listens on channel 1, that it answers.
- **Panic** sends "all sound off", "all notes off" and "sustain pedal up" on every channel, for a stuck note. Devices that only send MIDI, such as a keyboard, have neither button.
- **Forget** appears on a device that is not plugged in. It deletes its connections, name and settings. *Forget all* clears every unplugged device. A device you plug in again afterwards starts over as a new device.
- The **Network ports** and **Bluetooth** cards show the network MIDI endpoints (with their UDP ports and connected peers) and the Bluetooth MIDI port. *Add a network port* creates an extra, standalone network MIDI port with a name you choose, for example a bus: connect your computer to it once, then route it to several devices in the matrix. *Remove* deletes such a port and its connections. The network ports of USB devices are controlled by the device's *Network port* switch.

Console

The Console tab shows the MIDI traffic and lets you send MIDI by hand. On the right is a live list of the messages passing through from the moment you open the tab (the latest 500 rows), with time, direction, port, a readable description and the raw bytes. **rx** is what a port received from its device, **tx** is what was sent out to it (with the source after the arrow; what you send from this page is marked *this page*).



The Console tab.

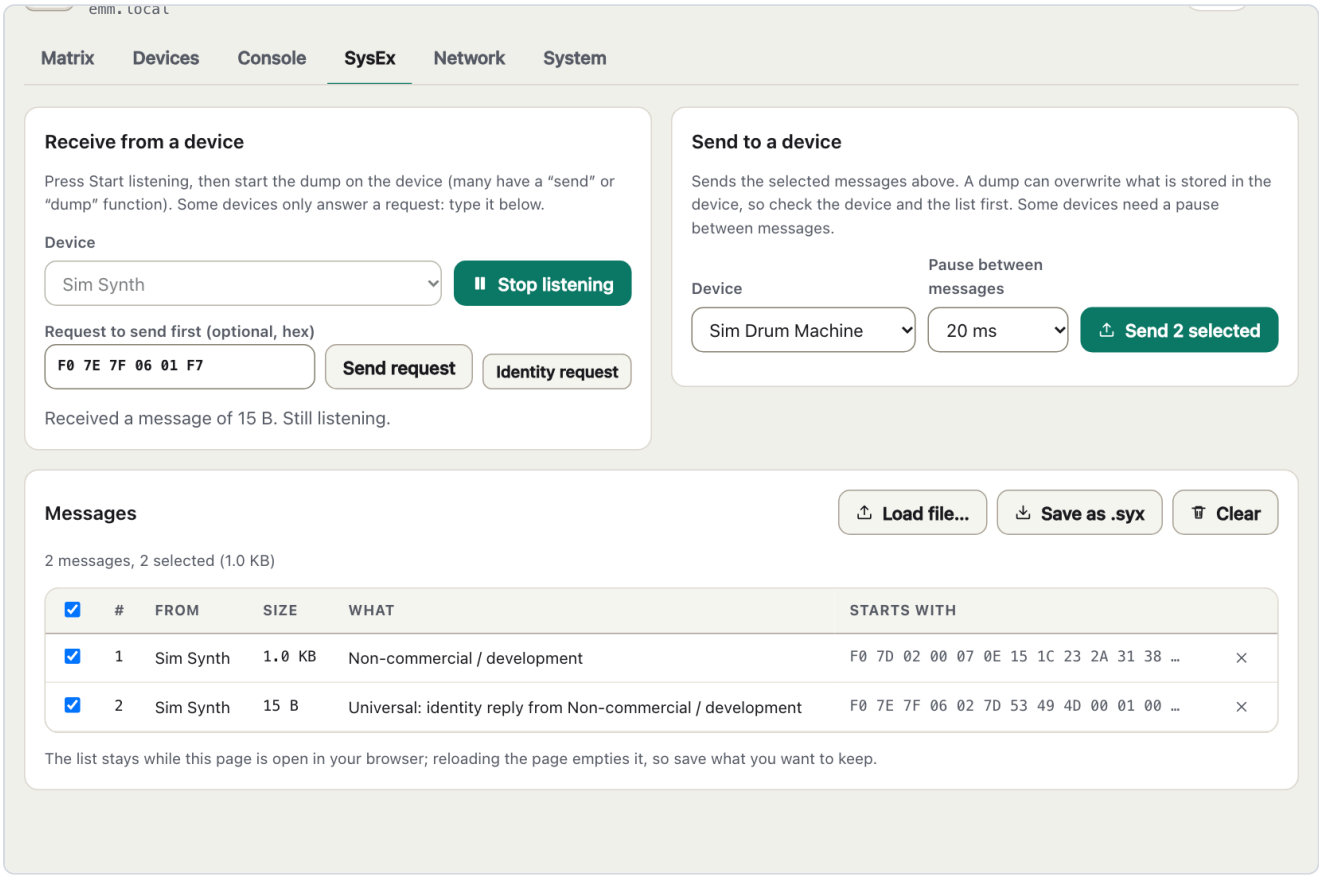
On the left you choose where to send: **Send to** lists the devices that are plugged in with USB and can receive MIDI, and **Channel** is used by everything below that needs one. Network and Bluetooth ports are not offered here. **Panic** sends "all sound off", "all notes off" and "sustain pedal up" on every channel.

- **Keyboard.** Two octaves and the top C. Press, tap or slide across the keys; the *Octave* buttons move the keyboard up or down one octave (middle C is C4), and the slider sets the velocity. A note that is held is let go when you release the key, switch to another tab or leave the window.
- **Common messages.** Sliders for modulation (controller 1), volume (7), pan (10), expression (11), pitch bend (it springs back to the middle when let go) and channel aftertouch. A program change with an optional bank (controllers 0 and 32), with – and + to step through programs. Buttons for all notes off, all sound off, reset all controllers, sustain pedal, and Start, Continue, Stop, Tune request and System reset (it asks first, because some devices restart or go back to factory settings). The sliders start at fixed positions: they do not show what the device currently has.

- **Custom message.** Pick a message type and its values and the bytes are shown before you send them, or type *raw bytes* in hex, for example `B0 07 64` or `F0 7D 01 F7`. A message must be complete: a missing byte or a stray one is refused rather than sent.
- The log has a **Port** menu that filters to one port: that port's rx and tx rows, plus the tx rows that came from it. Each entry ends with its type, for example *(USB)* or *(RTP-MIDI)*, because a device's USB port and its network port have the same name. **Direction** shows rx, tx or both. **Pause** freezes the list; **Clear** empties it. When you scroll up, auto-scrolling stops; *Jump to live* brings it back.

SysEx

SysEx (system exclusive) messages carry patch, setup and sample dumps that belong to one make of device. This tab receives such a dump into the browser, saves it as a `.syx` file, and sends a dump, from the browser or from a file, to a device. It works with USB devices only: a dump has to arrive complete, which network and Bluetooth MIDI cannot promise.

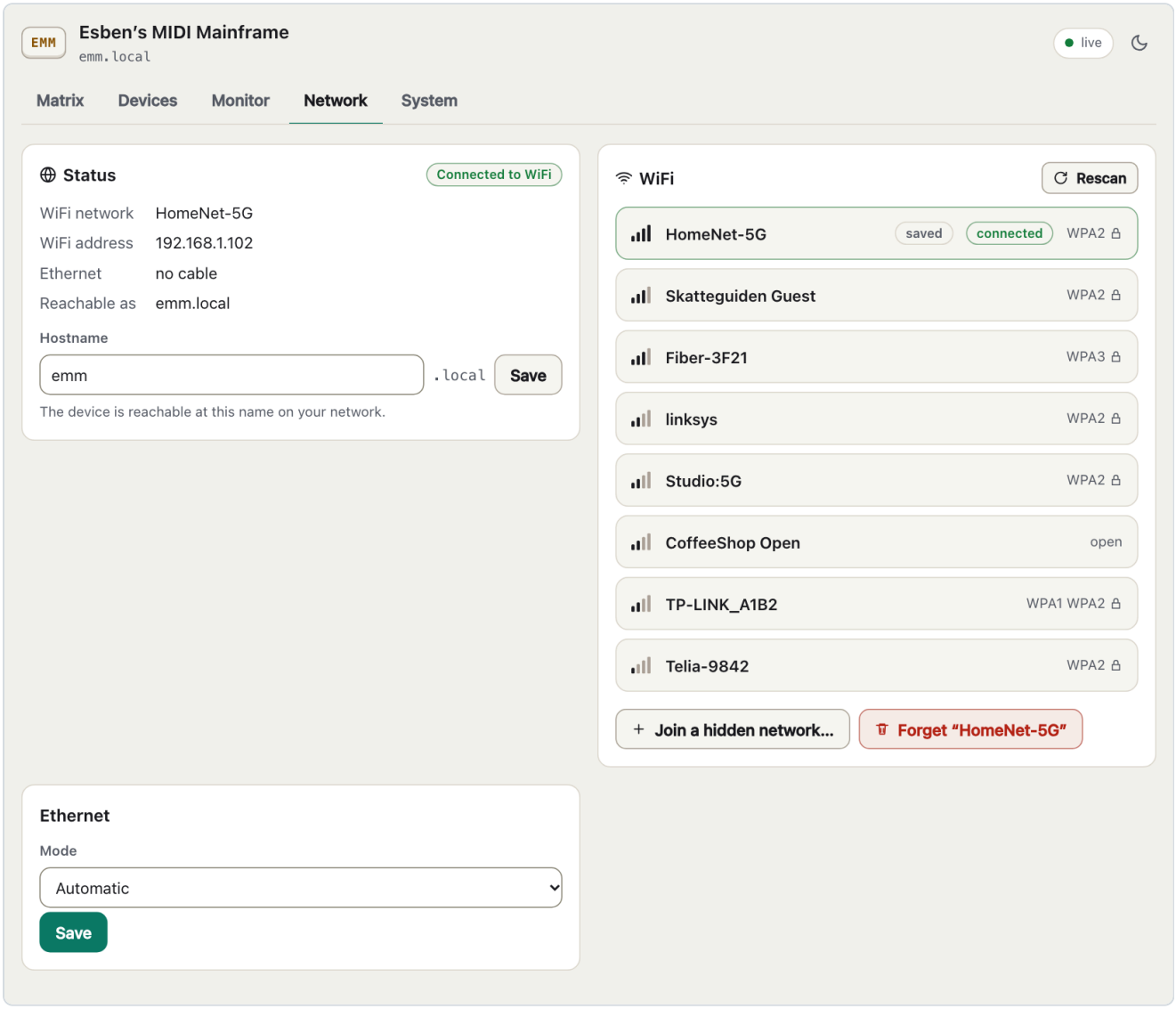


The SysEx tab, after a dump from a simulated synth.

- **Receive from a device.** Choose the device, press *Start listening* and start the dump on the device (many have a "send" or "dump" function). Some devices only answer a request: type it in hex under *Request to send first* and press *Send request*; *Identity request* fills in the standard "who are you" message. Every message that arrives is added to the list, with its size and, where it can be told, who made it. A message that stopped before its end (F7) is marked *cut short* and cannot be sent.
- **Messages.** The list holds what you received and the files you loaded (*Load file…*, or drop files on the list). Tick what you want and press *Save as .syx* to download it as one file. The list is kept in the browser while the page is open, also when you visit other tabs; reloading the page empties it, so save what you want to keep. A file or message can be up to 16 MB.

- **Send to a device.** Sends the ticked messages. Some devices need a pause between messages: choose it next to the device. A progress bar follows the transfer and *Cancel* stops it after the message that is being sent. Sending a dump can overwrite what the device has stored, so check the device and the list first.
- Listening does not interrupt the routing: what the device sends still goes on to wherever the matrix sends it.

Network



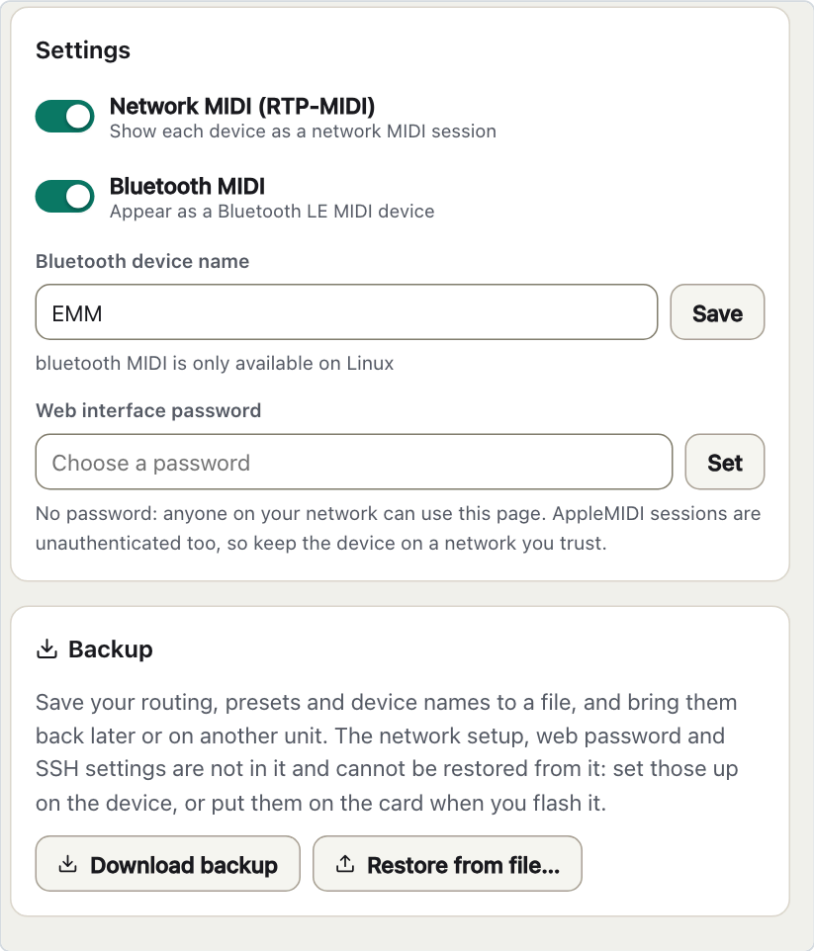
The Network tab.

- **Status** shows the WiFi network, the addresses, whether an Ethernet cable is connected, and the name the device answers to. Change the **hostname** here (letters, digits and hyphens; the device is then `name.local`).
- **WiFi** lists the networks in range with their signal strength. The one you are on is marked *connected*. Click a network to join it (it asks for the password). *Rescan* refreshes the list; *Join a hidden network* takes a name you type. *Forget* removes a network saved through EMM; if Ethernet is not connected the device then returns to the setup network and this page stops answering. A connection that was set up outside EMM has no Forget button.

- **Ethernet**: Automatic, DHCP, a static address (with gateway and DNS), or disabled.
- Anything fixed in `emm.txt` is shown as locked with an *emm.txt* mark, because the file wins at every boot.

> While the device is in setup mode, a rescan switches the setup network off for a few seconds. Your phone may lose its connection to it and should reconnect by itself.

System



The System tab (settings and backup). The Bluetooth message comes from the development machine used for the screenshot.

- **This device**: model, uptime, how many USB devices, CPU temperature, and whether the settings storage is writable. If it says the storage is read-only or unreadable, changes will not survive a restart.
- **Settings**: switch network MIDI (RTP-MIDI) and Bluetooth MIDI on or off; set the Bluetooth name (up to 20 characters); set a **web interface password** (at least 4 characters; *Remove* clears it, but not while SSH is on). Settings fixed in `emm.txt` are shown locked.
- **SSH access** (appliance card only). Off by default, and nothing listens on port 22 while it is off. Set a web password first (SSH is a root shell, so the switch stays locked until you do), then give SSH a password of 8 or more characters and/or paste public keys, and switch it on: `ssh root@emm.local` (the card shows the exact command, with your hostname). It stays on across reboots until you switch it off. On Raspberry Pi OS use the system's own `sshd` instead.
- **Backup**: see section 7.

- **Power** (shown on the appliance card and where the installer allowed it): reboot or shut down.

6. Connecting a computer, phone or tablet

Network MIDI (RTP-MIDI)

Every USB device, and every network port you add, is announced on the network as an RTP-MIDI (AppleMIDI) session. Your computer sees the synths as MIDI devices without any cable. Discovery uses Bonjour/mDNS, the same mechanism as `emm.local`.

System	How
macOS	Open *Audio MIDI Setup*, then *Window → Show MIDI Studio*, and double-click *Network*. Make sure your own session (*Session 1*) is ticked *Enabled*. EMM's devices appear in the *Directory* list; select one and click *Connect*. The device then shows up as a MIDI port in your music software.
Windows	Windows has no built-in RTP-MIDI. Install Apple's Bonjour (it comes with iTunes or Bonjour Print Services) and an RTP-MIDI driver such as rtpMIDI, then connect to the EMM sessions there.
Linux	Use an RTP-MIDI client that supports AppleMIDI sessions, with `avahi` for discovery.
iPhone, iPad	Apps with network MIDI support list the sessions; see the app's own MIDI settings.

Not verified yet: EMM's RTP-MIDI and Bluetooth MIDI have so far been tested only against EMM's own test code, not against Apple's, Windows' or Linux network MIDI software, nor against macOS, iOS or Android Bluetooth MIDI. The steps above are what should work, not a promise. If a computer cannot see or connect to a session, try another client, or Ethernet instead of WiFi, and see Troubleshooting.

A connection from the network behaves like any other port: in the matrix the network ports sit in their own group, so you can route a synth's output to a network port or the reverse, with filters.

Bluetooth MIDI

EMM appears as a Bluetooth LE MIDI device named `EMM` (or the name you chose); no PIN is needed. Connect from the MIDI application, not only from the system's Bluetooth list: on macOS *Audio MIDI Setup → Window → Show MIDI Studio → Bluetooth*; on iPhone and iPad in the Bluetooth MIDI settings inside apps such as GarageBand. Bluetooth MIDI has no cable numbers, so EMM shows it as one port pair, in the network group of the matrix. It suits notes and controllers. Keep the Bluetooth name short (about eight characters): a longer name may be cut in your device's list.

What network and Bluetooth MIDI are not for

Do not rely on network or Bluetooth MIDI for backups, restores or firmware updates

RTP-MIDI runs over UDP on a network that may drop or delay packets, and over WiFi it has no guarantee about delay and jitter. EMM does not implement the recovery journal of the RTP-MIDI standard (RFC 6295). When a packet is lost, or a session ends, EMM releases the notes that peer was holding so nothing sticks (a held sustain pedal is the exception: use *Panic* on the Devices tab), but a lost packet can cut a note the player is still holding. A SysEx message that loses a packet is discarded rather than forwarded, so a dump can arrive incomplete, with no error to tell you.

Good for: playing music, controlling a synth from a computer, and loading a patch now and then.

Not for: backing up or restoring a device, firmware updates, and anything where a single damaged message can do harm. A failed firmware update can leave a device unusable. For those, unplug the device from EMM's hub, connect it directly to your computer with a USB cable, and use the maker's own software. Bluetooth MIDI is for notes and controllers only; do not send SysEx dumps or samples over it.

Ethernet or 5 GHz WiFi is less bad than 2.4 GHz WiFi for a one-off patch or sample, but it does not make backups, restores or firmware updates safe.

7. Presets, backups and moving devices

Presets

Presets are described in section 5. They are stored on the device and are part of a backup.

Backup and restore

System tab → Backup. *Download backup* saves a file with your routing, presets, per-port names and settings, the list of devices EMM remembers, and the network MIDI and Bluetooth switches. *Restore from file…* puts a backup back after a confirmation, **replacing** the current routing, presets and names. A restore applies the connections like a preset recall: connections to devices that are in the backup but not plugged in are kept and show as *Waiting for device*; connections to a port the backup does not list, or that no longer has that direction, are skipped and reported.

> **What a backup does not contain**
> The **network setup (WiFi password), the web interface password and the SSH settings are not in a backup, and a restore never changes them**, not even from a hand-edited file. They are the device's own and are kept in a separate file, `system.json`. A backup file can therefore be shared or stored without exposing them.

Setting up a fresh card. Put the WiFi, hostname and web password in `emm.txt` before the first start, or copy `system.json` from another unit's card into an `emm` folder on the new card (create the folder; on Raspberry Pi OS it is `/var/lib/emm/system.json`). SSH is switched on in the web page; to keep its host key, so the fingerprint does not change, copy the `emm/ssh` folder as well. Then restore your backup from the System tab. A copied `system.json` includes the hostname, so do not run the old and the new unit on one network with the same name.

This backup is for EMM's own settings. It is **not** a backup of the synths: for those, use each device's own software with the device connected directly to your computer by USB (see section 6).

Moving a device to another hub port

A USB device that reports no serial number is told apart by the hub port it sits in. When you move one to another port, EMM carries its connections, name and network port over to the new port, as long as that is unambiguous: exactly one device of that model is attached, and exactly one is remembered as absent. With two identical devices EMM cannot tell them apart, so give each its own port and leave it there. Devices that do report a serial number are recognised wherever you plug them.

8. Security

EMM is made for a trusted home or studio network. Read this before putting it on a shared one.

- **The web interface has no password by default.** Anyone on the network can reroute MIDI and send SysEx to your synths. Set a password in the System tab or with `web_password` in `emm.txt`. It uses HTTP Basic authentication, so it is only worthwhile on a trusted network.
- **RTP-MIDI has no authentication at all.** Anyone on the network can connect to a network MIDI port and send SysEx to the synth behind it.
- **SSH** (appliance card) is a root shell. It is off at every boot unless you saved it on, and it cannot be switched on until the web interface has a password.
- **Bluetooth MIDI has no authentication either:** anyone within radio range can connect and send MIDI, including SysEx. Switch it off (System tab, or `ble = off`) if that matters.
- The **setup network is open** unless `ap_password` is set; the home WiFi password you type there crosses the air once. While it is up, anyone who joins it can change the WiFi settings, even if a web password is set. Set `ap_password` if that matters.
- **The WiFi password is stored in clear text** in `system.json` and `emm.txt` on the card. Treat the SD card like the password it contains.
- The server rejects requests for unknown host names (protection against DNS rebinding) and state-changing requests from other web sites. If you reach the device through a name of your own, add it with `allowed_hosts`.

9. Troubleshooting

emm.local does not open

It relies on mDNS. macOS, iOS and Linux (with `avahi`) resolve it; Windows 10/11 usually does. On a VPN or managed network, or on Android, use the IP address: look for `emm` in your router's client list, or `ping emm.local` from a computer where it works.

No WiFi at all, or the setup network never appears

Set `wifi_country` in `emm.txt`. EMM does not open the setup network while Ethernet has a working connection. On Raspberry Pi OS check the service with `systemctl status emmd` and `journalctl -u emmd -b`.

The setup page does not pop up on my phone

Open `http://emm.local/` or `http://10.42.0.1/` by hand. Turn off a VPN, and "private DNS" on Android.

A USB MIDI device does not show up

- Use a **powered** hub and the Pi's proper power supply. An under-voltage warning is the first thing to rule out.
- Only **class-compliant** devices work.
- On boards where a port can also act as a USB device (Zero, 3, CM4) make sure no gadget setup (`dtoverlay=dwc2`, `g_ether`) is active.
- On Raspberry Pi OS, `sudo emmd --list-devices` shows what was detected. On the appliance card the matrix shows what was found, and a monitor on the HDMI port shows the boot messages.

Bluetooth MIDI will not pair

Connect from the MIDI application, not only from the system Bluetooth list (section 6). Windows and Android need an app that supports Bluetooth LE MIDI. On 2.4 GHz, WiFi and Bluetooth share the radio: use 5 GHz WiFi or Ethernet if connections are unstable.

Transfers are slow or stall

Use Ethernet or 5 GHz WiFi, and a USB cable for anything large. RTP-MIDI over WiFi was not built for bulk data; the USB side is not the bottleneck.

The page says the settings storage is read-only or unreadable

Changes will not survive a restart. On the appliance card check that the SD card is not write-protected or worn out. A file that cannot be read is left untouched rather than overwritten, so a flaky card cannot wipe your settings.

I forgot the web password

Switch the Pi off and put the card in a computer. Either add `web_password = something-new` to `emm.txt` (it is used instead of the saved one), or delete `emm/system.json` and any `system.json.new` and `.old` next to it (this also clears the network and SSH settings, which you then set up again). Put the card back

and power on, then set a new password in the System tab and remove the line from `emm.txt` if you want to manage it there.

10. Reference

emm.txt keys

One `key = value` per line. Quote a value (`"…"` or `'…'`) if it contains `#` or starts or ends with a space; inside quotes everything is literal. A line EMM cannot use is ignored and the reason goes to its log.

Key	Values	Meaning
`hostname`	letters, digits, `-` ; up to 63 characters. Default `emm`	Reachable as `<hostname>.local`
`wifi_country`	two letters, e.g. `DK`	WiFi regulatory country; WiFi stays off in many regions until it is set
`wifi_ssid` , `wifi_password`	name up to 32 bytes; password 8–63 characters or 64 hex digits	WiFi network to join. An empty password means an open network. A password of the wrong length is tried anyway and the join fails
`ethernet`	`auto` , `dhcp` , `static` , `off`	Wired network mode. Default `auto`
`eth_address` , `eth_gateway` , `eth_dns`	`192.168.1.50/24` ; an address; one or more addresses	Used only with `ethernet = static`
`ap_ssid` , `ap_password`	name up to 32 bytes; password 8–63 characters or 64 hex digits	The setup network. Default `EMM-Setup-XXXX` , no password. A password of the wrong length is ignored and the setup network is then **open**
`rtp`	`on` / `off`	Network MIDI. Default on
`ble` , `ble_name`	`on` / `off` ; text	Bluetooth MIDI and the name shown when pairing. Default on, `EMM`
`web_password`	text	Password for the web interface; used instead of one saved from the web page
`allowed_hosts`	names separated by commas, spaces or semicolons; a leading dot allows a whole domain	Extra names the web interface accepts, for example a name your router gives the Pi

Passwords in this file are stored as plain text.

Files on the card

File	What
`emm.txt`	Your hand-written settings (optional). Never overwritten by EMM or an update
`reset-network`	Create it empty to get the setup network on the next boot; EMM deletes it

`emm/state.json`	Routing, presets, device names, the RTP and Bluetooth switches (created by EMM)
`emm/system.json`	Network (including the WiFi password), web password hash and SSH settings (created by EMM). Secret: not part of a backup
`emm/ssh/`	The SSH host key, so the fingerprint survives reboots (appliance card)
`*.new` , `*.old`	Leftovers of an interrupted save. EMM settles them at the next start (and keeps the newest complete copy)
`*.corrupt-…`	A damaged file that EMM set aside. It stays until you delete it, and may contain your WiFi password

> EMM saves its files so that a power cut at any moment leaves a complete copy: it writes a new, checksummed file first, swaps it in, and at the next start picks the newest complete copy. A damaged file is kept next to the new one, never silently deleted.

Limits and what is not included

- MIDI 1.0 only: USB MIDI 2.0 / UMP devices are not handled (their legacy endpoints work).
- No 5-pin DIN MIDI ports. A USB MIDI interface with DIN connectors is the practical answer.
- No RTP-MIDI recovery journal (see section 6).
- No encryption of the stored WiFi password; no authentication on RTP-MIDI.

> **Status**
> EMM is new. Most of it is covered by automated tests on a development machine with simulated devices. It has run on real Raspberry Pi hardware with real USB synths, and the appliance card has been started on a Raspberry Pi 3 B+ and a Pi 4 (a WiFi join problem found there is fixed in the card's kernel, not yet tested on a Pi). **Not verified yet:** the Pi 5, 500 and compute-module builds, the Bluetooth MIDI server, the SSH switch, saving behaviour on a real card after a power cut, and compatibility with other makers' RTP-MIDI and Bluetooth MIDI software. Expect rough edges, and use `emmd --log-level debug` when something does not behave.

Words used in this manual

Word	Meaning
Appliance card	The ready-to-run SD card made by the project: a small system that runs from memory
DHCP	Your router hands out the device's network address automatically
FAT32, MBR	The disk format and partition table the Pi's bootloader can read
mDNS, Bonjour, avahi	How computers find `emm.local` and network MIDI sessions without you typing an address
OTG	A USB port that can act as host or as device (Pi Zero 2 W, compute modules)

RTP-MIDI, AppleMIDI	MIDI over a network, also called network MIDI
SysEx	System exclusive: long MIDI messages used for patch dumps, backups and firmware
UMP	Universal MIDI Packet, the format of USB MIDI 2.0